

HOWARD Help Calculations

1 Calculations

List of available calculations

1.1 BARCODE

BARCODE as VaRank tool

BARCODE as VaRank tool. This calculation generates a BARCODE for each sample based on sample genotype (0 for reference, 1 for heterozygous, 2 for homozygous alternate).

1.2 BARCODEFAMILY

BARCODEFAMILY as VaRank tool

BARCODEFAMILY as VaRank tool. This calculation generates a BARCODE for each family based on sample genotype (0 for reference, 1 for heterozygous, 2 for homozygous alternate) and family relationships.

1.3 COUNT_SAMPLES

Number of sample that have a genotype for the variant (for multi sample VCF)

Number of sample that have a genotype for the variant (for multi sample VCF). This calculation counts the number of samples (fixed 'count_samples' field) that have a genotype for a given variant in a multi-sample VCF file. It helps in assessing the presence of the variant across different samples.

1.4 DP_STATS

Depth (DP) statistics

Depth (DP) statistics. This calculation provides statistical measures for the sequencing depth (DP) across different samples. It helps in understanding the coverage and reliability of variant calls.

1.5 FIND_SAMPLES

Number of sample that have a genotype for the variant (for multi sample VCF)

This calculation counts the number of samples that have a genotype for a given variant in a multi-sample VCF file. It helps in assessing the presence of the variant across different samples.

Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see [help.parameters.pdf](#)). The parameter 'tags' allows for the specification of the INFO tags to be created, with the key being the tag name and the value being either 'count' or 'list' to indicate whether to count the samples or list them. For example, to create an INFO tag 'count_samples' that counts the number of samples and an INFO tag 'list_samples' that lists the samples, you can specify:

```
{ "tags": {"count_samples": "count", "list_samples": "list"} }
```

Otherwise, change the default tags in the parameter file, like in this example:

```
{ "tags": {"count_samples_for_variants": "count", "list_samples_for_variants": "list", "another_list_samples_
```

1.6 GENOTYPECONCORDANCE

Concordance of genotype for multi caller VCF

Concordance of genotype for multi caller VCF. This calculation assesses the concordance of genotypes for a given variant across multiple callers in a multi-caller VCF file. It helps in evaluating the consistency of genotype calls and can be used to identify variants with high confidence based on agreement among different callers.

1.7 LIST_SAMPLES

List of samples that have a genotype for the variant (for multi sample VCF)

List of samples that have a genotype for the variant (for multi sample VCF). This calculation provides a list of samples (fixed 'list_samples' field) that have a genotype for a given variant in a multi-sample VCF file. It helps in understanding which samples support the presence of the variant.

1.8 MERGED_HGVS

Merge HGVS nomenclatures from snpEff (snpeff_hgvs) and ANNOVAR (AChange_refGene) into merged_hgvs field

This calculation merges HGVS nomenclatures from snpEff (snpeff_hgvs) and ANNOVAR (AChange_refGene) into a single field called merged_hgvs. It aggregates distinct HGVS annotations from both sources, concatenating them with a comma separator. This unified field allows for easier access to HGVS information regardless of the annotation source, facilitating downstream analysis and interpretation.

1.9 NOMEN

NOMEN information (e.g. NOMEN, CNOMEN, PNOMEN...) from HGVS nomenclature field (see parameters help)

Extract NOMEN information (e.g. NOMEN, CNOMEN, PNOMEN...) from HGVS nomenclature field (see parameters help). This calculation parses the HGVS nomenclature field to extract specific NOMEN information such as NOMEN, CNOMEN, PNOMEN, etc., based on the parameters provided. It creates new INFO fields with the extracted NOMEN information for easier access and downstream analysis.

1.10 NOMEN_SNPEFF

NOMEN information (e.g. NOMEN, CNOMEN, PNOMEN...) from HGVS nomenclature field (see parameters help)

Extract NOMEN information (e.g. NOMEN, CNOMEN, PNOMEN...) from HGVS nomenclature field (see parameters help). This calculation parses the HGVS nomenclature field to extract specific NOMEN information such as NOMEN, CNOMEN, PNOMEN, etc., specifically based on snpEff HGVS annotations field 'snpeff_hgvs'. It creates new INFO fields with the extracted NOMEN information for easier access and downstream analysis.

1.11 RECREATE_INFO_FIELDS

Recreate INFO_tags, rename or remove tags

Recreate INFO_tags, rename or remove tags. This calculation allows for the recreation of INFO fields by renaming existing tags or removing them based on specified parameters. It provides flexibility in managing INFO fields, enabling users to customize their VCF annotations according to their analysis needs.

1.12 RENAME_INFO_FIELDS

Rename or remove INFO/tags

Rename or remove INFO/tags. This calculation allows for the renaming of existing INFO fields or the removal of specific tags based on provided parameters. It helps in standardizing or customizing INFO fields in the VCF according to user requirements and facilitates downstream analysis. Use parameter section 'calculation_RENAME_INFO_FIELDS' in param.json to specify the INFO fields to rename or remove.

This example will rename INFO field 'ENOMEN' to 'exon' and remove INFO fields 'SPiP', 'SPiP_Alt' and 'SPiP_distSS' from the 'variants' table:

```
{"fields_to_rename": {"ENOMEN": "exon", "SPiP": null, "SPiP_Alt": null, "SPiP_distSS": null }, "table": "vari
```

1.13 SNPEFF__ANN__EXPLODE

Explode snpEff annotations

Explode snpEff annotations from the ANN field, creating new INFO fields with prefix 'snpeff__' for each annotation. This calculation takes the ANN field from snpEff annotations, which may contain multiple annotations for a single variant, and explodes it so that each annotation is represented in separate INFO fields with a 'snpeff__' prefix. This allows for easier access to individual annotations and facilitates downstream analysis.

1.14 SNPEFF__ANN__EXPLODE__JSON

Explode snpEff annotations in JSON format

Explode snpEff annotations from the ANN field, creating a new INFO field 'snpeff__json' in JSON format that contains all annotations. This calculation takes the ANN field from snpEff annotations, which may contain multiple annotations for a single variant, and explodes it into a structured JSON format stored in a new INFO field named 'snpeff__json'. This allows for easier access to all annotations in a single field and facilitates downstream analysis.

1.15 SNPEFF__ANN__EXPLODE__UNIQUIFY

Explode snpEff annotations with unify values

Explode snpEff annotations from the ANN field, creating new INFO fields with prefix 'snpeff__uniquify__' for each annotation and ensuring unique values. This calculation takes the ANN field from snpEff annotations, which may contain multiple annotations for a single variant, and explodes it so that each annotation is represented in separate INFO fields with a 'snpeff__uniquify__' prefix, ensuring that only unique values are retained. This allows for easier access to individual annotations and facilitates downstream analysis.

1.16 SNPEFF__EXTRACT

HGVS nomenclatures from snpEff annotation

Extract HGVS nomenclatures from snpEff annotation (field ANN) and create new INFO fields with prefix 'snpeff__' (e.g. snpeff__hgvs, snpeff__impact, snpeff__gene_name...). This calculation parses the ANN field from snpEff annotations, extracting relevant information such as HGVS nomenclatures, impact, gene name, etc., and creates new INFO fields with a 'snpeff__' prefix for easier access and downstream analysis.

1.17 SNPEFF__HGVS

HGVS nomenclatures from snpEff annotation

Extract HGVS nomenclatures from snpEff annotation (field ANN) and create new INFO field 'snpeff__hgvs'. This calculation specifically targets the extraction of HGVS nomenclatures from the ANN field of snpEff annotations, creating a new INFO field named 'snpeff__hgvs' that contains the extracted HGVS information for easier access and downstream analysis.

1.18 TRANSCRIPTS__ANNOTATIONS

Perform transcripts annotations and generate a transcripts table/view (using JSON parameters file)

Perform transcripts annotations and generate a transcripts table/view. This calculation allows for the inclusion of transcript annotations in both JSON and structured formats, providing flexibility in how transcript information is stored and accessed within the variant analysis pipeline. The specific format(s) used can be determined based on parameters specified in the JSON parameter file in 'transcripts' section (see help.parameters.pdf), or directly in the calculation parameters.

1.19 TRANSCRIPTS__JSON

Perform transcripts annotations and export into INFO field in JSON format (field 'transcripts__json')

Perform transcripts annotations and generate a transcripts table/view. This calculation allows for the inclusion of transcript annotations in both JSON and structured formats, providing flexibility in how transcript information is stored and accessed within the variant analysis pipeline. The specific format(s) used can be determined based on parameters specified in the JSON parameter file in 'transcripts' section (see help.parameters.pdf). Then and add transcripts fields into INFO fields in structured format (field 'transcripts__ann'). This calculation allows for the inclusion of transcript annotations in a JSON format, facilitating the integration of transcript information into the variant analysis pipeline.

1.20 TRANSCRIPTS__ANN

Perform transcripts annotations and export into INFO field in structured format (field 'transcripts__ann')

Perform transcripts annotations and generate a transcripts table/view. This calculation allows for the inclusion of transcript annotations in both JSON and structured formats, providing flexibility in how transcript information is stored and accessed within the variant analysis pipeline. The specific format(s) used can be determined based on parameters specified in the JSON parameter file in 'transcripts' section (see help.parameters.pdf). Then and add transcripts fields into INFO fields in structured format (field 'transcripts__ann'). This calculation allows for the inclusion of transcript annotations in a structured format, facilitating the integration of transcript information into the variant analysis pipeline.

1.21 TRANSCRIPTS__EXPORT

Export transcripts table/view as a file (using JSON parameters file)

Export transcripts table/view as a file. This calculation allows for the export of the transcripts table or view generated from transcript annotations to an external file format such as TSV or CSV. The parameters for this calculation can be specified in the JSON parameter file, either in 'transcripts' section or directly in the calculation parameters (see help.parameters.pdf), including options for the output file path, format, and any filters to apply during export.

1.22 TRANSCRIPTS__PRIORITIZATION

Prioritize transcripts with a prioritization profile (using JSON parameters file)

Prioritize transcripts with a prioritization profile. This calculation allows for the prioritization of transcripts based on a predefined profile specified in the JSON parameter file, either in 'transcripts' section or directly in the calculation parameters (see help.parameters.pdf), helping to identify the most relevant transcripts for further analysis.

1.23 TRANSCRIPTS__PRIORITIZATION__STRICT

Prioritize transcripts with a prioritization profile (using JSON parameters file)

Prioritize transcripts with a prioritization profile, ensuring that all needed annotations are present and considered. This calculation allows for the prioritization of transcripts based on a predefined profile specified in the JSON parameter file, either in 'transcripts' section or directly in the calculation parameters (see help.parameters.pdf), helping to identify the most relevant transcripts for further analysis.

1.24 TRIO

Inheritance for a trio family

Inheritance for a trio family. This calculation assesses the inheritance pattern of a variant within a trio family pedigree (father, mother, and child). It helps in understanding the transmission of genetic variants and identifying potential de novo mutations. Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see help.parameters.pdf). The parameter 'trio_samples' allows for the specification of the sample names for father, mother, and child:

```
{"father": "sample_father", "mother": "sample_mother", "child": "sample_child"}
```

This parameter can also be specified in a JSON file (containing the trio sample names):

```
"/path/to/trio_samples.json"
```

If no pedigree information is provided, the calculation will use the first 3 samples in the VCF file.

1.25 VAF

Variant Allele Frequency (VAF) harmonization

Variant Allele Frequency (VAF) harmonization. This calculation normalizes the Variant Allele Frequency (VAF) across different samples and callers to ensure consistency in VAF representation. It helps in comparing VAF values across samples and callers by applying a harmonization process.

1.26 VAF_STATS

Variant Allele Frequency (VAF) statistics

Variant Allele Frequency (VAF) statistics. This calculation provides statistical measures for the Variant Allele Frequency (VAF) across different samples. It helps in understanding the distribution and variability of VAF values.

1.27 VARIANT_CHR_POS_ALT_REF

Create a variant ID with chromosome, position, alt and ref

This calculation generates a variant ID with chromosome, position, alt and ref, with format '#CHROM_POS_REF_ALT'. Useful for variant identification and comparison across datasets

1.28 VARIANT_ID

Variant ID generated from variant position and type

Variant ID generated from variant position and variation (ref and alt) and type (SVTYPE). This calculation creates a unique identifier for each variant, facilitating variant tracking and comparison across datasets. Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see [help.parameters.pdf](#)). The parameter 'variant_id_tag' allows for the specification of the INFO tag to be used for the variant ID. By default, it uses 'variant_id', but it can be changed to any other INFO tag name as needed. For example, to use 'varid' as the INFO tag for the variant ID, you can specify:

```
{ "variant_id_tag": "varid" }
```

Other options as 'variant_id_tag_info' allows for the specification of the description in the VCF header, and 'keep_variant_id_tag_column' allows for keeping the variant ID tag column in the 'variants' table for further analysis.

As an example of the JSON parameter file, you can specify:

```
{ "variant_id_tag": "varid", "variant_id_tag_info": "Variant ID generated from variant position and type", "keep_variant_id_tag_column": true }
```

1.29 VARIANT_ID_VARID

Variant ID generated from variant position and type, using 'varid' as INFO tag

Variant ID generated from variant position and variation (ref and alt) and type (SVTYPE). This calculation creates a unique identifier for each variant, facilitating variant tracking and comparison across datasets.

1.30 VARTYPE

Variant type (e.g. SNV, INDEL, MNV, BND...)

Determine the type of variant based on its characteristics, such as the length of the reference and alternate alleles, and the presence of structural variant information. This calculation classifies variants into types like SNV, INDEL, MNV, BND, etc., which is essential for downstream analysis and interpretation.